



**UNIVERSIDAD
CATÓLICA DE
TEMUCO**

**DEPARTAMENTO DE
INGENIERÍA INFORMÁTICA**
FACULTAD DE INGENIERÍA

**CÁLCULO DE EFECTOS OLVIDADOS CON MÚLTIPLES
EXPERTOS MEDIANTE PROCESAMIENTO TENSORIAL EN
PYTHON**

por

CLAUDIO ESTEBAN ARAYA TORO

Trabajo de Título presentado a la
Facultad de Ingeniería de la Universidad Católica de Temuco
Para Optar al Título de Ingeniero Civil Informático.

- Temuco, enero 2025 -



COMISION EXAMEN DE TITULO

Este Examen de Título ha sido realizado en el Departamento de Ingeniería Informática:

Presidente Comisión:

.....
Sr. Alejandro Mellado Gatica
Magister en Telecomunicaciones
Master en Soc. de la Información y el
Conocimiento
Master en Ciberseguridad y Privacidad
Ingeniero Informático
Jefe Carrera Ing. Civil Informática

Profesor Guía:

.....
Dr. Julio Rojas Mora
Ingeniero de Sistemas
Doctor en Estadística e Inv. Operativa
Director Departamento Ing. Civil Informática

Profesor Informante:

.....
Sr. Elliott Mardones Arias
Ingeniero Civil en Informática

Jefe Carrera de Ingeniería
Civil Informática:

.....
Sr. Alejandro Mellado Gatica
Magister en Telecomunicaciones
Master en Soc. de la Información y el
Conocimiento
Master en Ciberseguridad y Privacidad
Ingeniero Informático
Jefe Carrera Ing. Civil Informática

Temuco, enero 2025.



INFORME TRABAJO DE TITULO

TITULO : "CALCULO DE EFECTOS OLVIDADOS CON
MULTIPLES EXPERTOS MEDIANTE
PROCESAMIENTO TENSORIAL EN PYTHON"

ALUMNO : CLAUDIO ESTEBAN ARAYA TORO

En mi condición de Profesor Guía presento el informe de calificación del Trabajo de Título del estudiante bajo las siguientes observaciones:

- El trabajo presenta la implementación de un paquete para el cálculo del método de efectos olvidados.
- El paquete fue implementado en Python y TensorFlow, utilizando matemática tensorial altamente paralelizada para acelerar los tiempos de computo.
- Los resultados fueron validados contra el estado de computo.
- El trabajo realizado deja un camino para mejorar posteriores con factibilidad real de ser implementadas.

Bajo estas consideraciones califico el desarrollo de este Trabajo de Título con **nota 7,0 (siete coma cero)**.

Dr. Julio Rojas M.
Profesor Guía

Temuco, 20 de enero del 2025.



INFORME TRABAJO DE TITULO

TITULO : “CALCULO DE EFECTOS OLVIDADOS CON
MULTIPLES EXPERTOS MEDIANTE
PROCESAMIENTO TENSORIAL EN PYTHON”

ALUMNO : CLAUDIO ESTEBAN ARAYA TORO

En mi calidad de Profesor Informante mis apreciaciones del informe son las siguientes:

- Los resultados del paquete forgeffects muestran una notable mejora en los tiempos de cómputo, evidenciando la capacidad del estudiante para optimizar procesos complejos mediante técnicas avanzadas como el procesamiento tensorial.
- La publicación del paquete forgeffects en el repositorio PyPI, junto con una descripción detallada y una guía de uso, refleja el compromiso del estudiante con altos estándares profesionales y la calidad de su trabajo. Esta iniciativa facilita que su contribución sea accesible y replicable por otros profesionales, promoviendo la transparencia y la colaboración en los ámbitos académico y tecnológico.
- Finalmente, considero que el estudiante Claudio Esteban Araya Toro ha demostrado durante el desarrollo de este trabajo, así como en su trayectoria académica, poseer las competencias técnicas, analíticas y profesionales que lo habilitan como un Ingeniero Civil Informático capacitado para enfrentar desafíos en su campo.

De acuerdo a estas consideraciones califico el desarrollo de este Trabajo de Titulo con **nota 7,0 (siete coma cero)**.

In. Elliott Mardones A.
Profesor Informante

Temuco, 20 de enero del 2025.

AGRADECIMIENTOS

En primer lugar, quiero expresar mi más profundo agradecimiento a mi familia, especialmente a mis padres, cuyo amor, apoyo y ejemplo han sido fundamentales para alcanzar este logro. Y a mi pareja, por su comprensión, cariño y compañía en todo momento.

También agradecer a mi tutor de tesis el Dr. Julio Rojas, por su guía y apoyo a lo largo de este trabajo y por tener la paciencia de siempre explicarme las cosas incluso mas de dos veces para hacerlas lo mejor posible. De igual modo, agradecer al profesor Elliott Mardones, quien me dio la valiosa oportunidad de formar parte de este proyecto y me motivó siempre a investigar y a estudiar para crecer como profesional.

Finalmente, agradecer a mis amigos, quienes fueron de gran motivación para la finalización de este trabajo.

Índice general

Índice general	I
Índice de figuras	III
Índice de cuadros	IV
Resumen	V
Abstract	VI
1 Introducción	1
1.1. Problema	1
1.2. Estado del Arte	2
1.3. Objetivos	3
1.3.1. Objetivo general	3
1.3.2. Objetivos específicos	3
2 Marco teórico	5
2.1. Qué es un Tensor	5
2.2. Matrices de Incidencia	6
2.3. Efectos Olvidados	8
2.4. Efectos Directos	11
2.5. Validaciones	12
2.5.1. Evaluación y resultados esperados	12
3 Diagnóstico del problema	14
3.1. Rendimiento paquete forgotten	16
4 Diseño	18

4.1. Estructura del paquete	18
4.1.1. Cálculo de efectos olvidados	19
4.1.2. Cálculo de efectos directos	21
4.1.3. Conjuntos de datos incluidos en el paquete	22
4.1.4. Bibliotecas utilizadas	22
5 Implementación	25
5.1. Función de efectos olvidados - FE()	25
5.2. Función de efectos directos - directEffects()	30
6 Validación	32
6.1. Rendimiento paquete forgeffects	34
6.2. Resultados de efectos olvidados	37
6.2.1. Resultados de segundo orden	38
6.2.2. Resultados de tercer orden	39
6.3. Resultados de efectos directos	40
7 Trabajo Futuro	42
7.1. Optimización algoritmo efectos olvidados	42
7.2. Implementación interactiva para usuarios finales	43
8 Conclusiones	44
Bibliografía	45

Índice de figuras

2.1. Ejemplo de relaciones directas e indirectas.	9
2.2. Ejemplo de relaciones directas e indirectas con múltiples expertos.	10
3.1. Gráfico de rendimiento de efectos olvidados en foRgotten. . .	16
4.1. Representación gráfica de estructura de forgeffects.	24
5.1. Esquema de convolución max-min (Kaufmann & Gil Aluja, 1988, p. 28)	26
6.1. Gráfico de rendimiento de efectos olvidados en forgeffects.	34
6.2. Gráfico de rendimiento de efectos olvidados en foRgotten y forgeffects.	35

Índice de cuadros

2.1. Tabla endecadaria de grados de verdad.	6
2.2. Grafo bipartito encadenado	7
3.1. Valores de parámetros para las pruebas de rendimiento en la función de efectos olvidados en foRgotten	14
3.2. Especificaciones de hardware y software utilizados para este trabajo	15
3.3. Resultados pruebas de rendimiento en foRgotten.	16
5.1. Comparación entre boot.t.test en R y lo realizado en Python adaptado para efectos directos.	31
6.1. Valores de parámetros para las pruebas de rendimiento en la función de efectos olvidados en forgeffects	33
6.2. Resultados pruebas de rendimiento en forgeffects con CPU.	33
6.3. Resultados pruebas de rendimiento en forgeffects con GPU.	34
6.4. Tabla de contingencia de aparición de efectos olvidados de segundo orden entre foRgotten y forgeffects	36
6.5. Tabla de contingencia de aparición de efectos olvidados de tercer orden entre foRgotten y forgeffects	36
6.6. Valores de parámetros para las pruebas de validación en las funciones de efectos directos en foRgotten y forgeffects.	37
6.7. Tabla de contingencia de niveles de significancia (p -valores) en las funciones de efectos directos entre foRgotten y forgeffects.	37

Resumen

Este trabajo aborda la optimización del cálculo de la teoría de los efectos olvidados mediante el desarrollo de un paquete denominado `forgeeffects` para Python, el cual se basa principalmente en procesamiento tensorial. El enfoque propuesto busca superar las limitaciones de las herramientas existentes, mejorando tanto la eficiencia como la velocidad de los cálculos, con especial énfasis en la integración de múltiples expertos para modelar relaciones de causa y efecto. Además, el paquete incorpora la capacidad de calcular efectos directos como una extensión del análisis.

La solución utiliza tensores para realizar operaciones con cada experto, lo que permite agilizar significativamente los procesos computacionales. Para validar la efectividad del enfoque, se realizaron comparativas de rendimiento con su implementación anterior en el entorno de desarrollo R, evidenciando mejoras significativas en términos de escalabilidad y reproducibilidad.

Palabras Clave: Teoría de los efectos olvidados, procesamiento tensorial, efectos directos, Python.

Resumen

This work addresses the optimization of the calculation of the forgotten effects theory through the development of a package called `for effects` for Python, which primarily relies on tensor processing. The proposed approach seeks to overcome the limitations of existing tools by improving both the efficiency and speed of calculations, with a particular focus on integrating multiple experts to model cause-and-effect relationships. Additionally, the package includes the capability to calculate direct effects as an extension of the analysis.

The solution leverages tensors to perform operations with each expert, significantly streamlining computational processes. To validate the effectiveness of the approach, performance comparisons were conducted with its previous implementation in the R development environment, demonstrating significant improvements in scalability and reproducibility.

Keywords: Forgotten effects theory, tensor processing, direct effects, Python.

Capítulo 1

Introducción

1.1. Problema

La teoría de los efectos olvidados, introducida por (Kaufmann & Gil Aluja, 1988), se centra en la identificación de impactos indirectos en sistemas complejos donde las relaciones de causa-efecto no son inmediatamente evidentes o directas. En un sistema típico de análisis de causa-efecto, las relaciones directas son más fáciles de identificar y medir; sin embargo, los efectos olvidados son aquellos impactos indirectos y no triviales que emergen a medida que las interacciones entre múltiples factores se enlazan entre sí. Uno de los principales desafíos en el cálculo de efectos olvidados es la dificultad para medir y agregar la incidencia, una noción subjetiva que representa la influencia de un conjunto de entidades sobre otro. Problema que se complica aún más cuando la incidencia debe ser evaluada por múltiples expertos, cada uno de los cuales puede tener perspectivas, experiencias y niveles de incertidumbre diferentes.

Para abordar este problema, anteriormente se desarrolló un paquete en GNU R llamado *foRgotten*, creado por (Mardones-Arias & Rojas-Mora, 2022). Este paquete permite modelar cada relación de causa-efecto a través de un vector de incidencias, vector que podía ser generado por expertos o derivado de múltiples medidas realizadas. Aunque el paquete proporcionaba herramientas para el análisis de efectos tanto directos como indirectos, su implementación resultó ser insuficiente para manejar grandes volúmenes de datos debido a limitaciones en términos

de velocidad y capacidad computacional específicamente en los cálculos de efectos olvidados con remuestreos empíricos.

Para superar estas dificultades, se propone una solución que aprovecha las ventajas de procesamiento tensorial, una tecnología que permite manejar datos de manera más rápida y eficiente, lo cual resulta fundamental al integrar datos de varios expertos y abordar la complejidad de las relaciones indirectas en sistemas como este. El trabajo se lleva a cabo en el entorno de Python mediante un paquete llamado `forgeffects` utilizando Tensorflow (Abadi et al., 2015), que permite modelar las relaciones de causa-efecto de manera eficiente, utilizando tensores para representar la información proporcionada por múltiples expertos, tanto para el cálculo de efectos directos como indirectos.

1.2. Estado del Arte

La teoría de efectos olvidados ha sido aplicada ampliamente en distintos contextos a lo largo de los años, consolidándose como una herramienta versátil en la investigación y la toma de decisiones estratégicas. En el ámbito político (Kaufmann & Gil Aluja, 1988, pp. 105-126) la emplearon para analizar factores que influyen en los procesos electorales, con el propósito de optimizar la asignación de recursos financieros hacia acciones clave que potencien la visibilidad de candidatos o grupos específicos. Asimismo, en el ámbito financiero, (Kaufmann & Gil Aluja, 1988, pp. 127-151) demostraron su utilidad al agrupar elementos económicos de una empresa, facilitando decisiones dirigidas a fortalecer su imagen financiera, identificar factores críticos y mejorar la percepción de los agentes económicos.

Actualmente, uno de los usos que se le ha dado a la teoría de efectos olvidados es en el análisis de sistemas económicos complejos, como en el estudio de redes de comercio internacional realizado por (Chávez-Bustamante et al., 2023) donde se analiza el flujo comercial global utilizando datos de exportaciones con el objetivo de identificar efectos indirectos en las relaciones comerciales, es decir, cómo el comercio entre dos países puede ser influenciado por la interacción con un tercero, el cuál encontró que la red comercial no sigue una estructura clásica sino un patrón donde ciertas economías, como Estados Unidos, China

y Alemania, actúan como centros que conectan con otras regiones.

Dada la relevancia de estos estudios, las alternativas seleccionadas en este trabajo para optimizar el cálculo de efectos olvidados resaltan el uso de arquitecturas que puedan hacer frente a una solución eficiente. Existen enfoques que integran arquitecturas tensoriales para abordar problemas multidimensionales como puede ser el trabajo de (Drakopoulos et al., 2017), donde se propuso un modelo basado en un tensor de tercer orden para agrupar documentos biomédicos. Este modelo, centrado en la representación semántica de documentos mediante palabras clave, términos y relaciones documentales, utiliza técnicas avanzadas para descomponer y analizar grandes volúmenes de datos, el cual demuestra cómo los tensores permiten manejar la complejidad de datos de manera estructurada. Otro ejemplo es el trabajo de (Wang et al., 2022), donde los autores desarrollaron un marco de simulación de dinámica de fluidos computacional (CFD) basado en operaciones tensoriales utilizando TensorFlow, si bien este estudio lo realizaron principalmente con unidades de procesamiento tensorial (TPU) el enfoque que le dan demuestra el que las arquitecturas tensoriales pueden gestionar cálculos complejos de manera rápida y escalable.

1.3. Objetivos

1.3.1. Objetivo general

- Implementar una solución para el cálculo de efectos olvidados con múltiples expertos mediante procesamiento tensorial en Python.

1.3.2. Objetivos específicos

- **Diagnóstico del problema.** Identificar las limitaciones de velocidad y de capacidad computacional en el cálculo de efectos olvidados en el paquete `foRgotten` en R.
- **Diseño e implementación del problema.** Desarrollar una solución basada en procesamiento tensorial para mejorar la eficiencia del cálculo de los efectos olvidados. El sistema deberá ser capaz de procesar grandes volúmenes de datos de múltiples expertos,

gestionando un número considerable de relaciones de causa-efecto de manera más rápida que la solución actual.

- **Validación de la implementación.** Validar la eficacia de la implementación de procesamiento tensorial para el cálculo de efectos olvidados, evaluando el rendimiento, precisión y escalabilidad en comparación con la solución original en R.

Capítulo 2

Marco teórico

2.1. Qué es un Tensor

Un tensor es una generalización de los conceptos de escalares, vectores y matrices a un objeto multidimensional. En si, los tensores permiten representar datos y relaciones complejas en múltiples dimensiones. En este contexto, los tensores se utilizan para modelar relaciones de incidencia entre diferentes conjuntos de datos.

Un tensor tridimensional se puede interpretar como una colección ordenada de matrices. Cada matriz dentro del tensor puede representar distintos expertos, y las demás dimensiones modelan las interacciones entre causas (c) y efectos (e) de la forma (k, c, e) , donde:

- k representa el número de matrices.
- c corresponde al número de filas en cada matriz
- e corresponde al número de columnas en cada matriz.

El tensor entonces nos va a permitir capturar de manera compacta información en estas tres dimensiones, siendo útil en aplicaciones que requieren modelar relaciones complejas, como las que presentamos en este trabajo.

2.2. Matrices de Incidencia

Para crear una matriz de incidencia, es fundamental comprender cómo se estructuran las relaciones de incidencia. Definimos un tensor tridimensional T_k como un grafo multiplex, que es una representación de red en la que múltiples capas representan diferentes tipos de interacciones o relaciones entre los mismos nodos (A. Alves et al., 2018). En este caso, cada k representa un «experto».

$$T = \{T_k\}_{k=1}^n$$

T_k puede ser un tensor de tamaño $c \times c$ (causa-causa), $e \times e$ (efecto-efecto) o $c \times e$ (causa-efecto).

Grados de Verdad	Evaluación
0	Falso
0,1	Prácticamente falso
0,2	Casi falso
0,3	Bastante falso
0,4	Más falso que verdadero
0,5	Ni verdadero ni falso
0,6	Más verdadero que falso
0,7	Casi verdadero
0,8	Bastante verdadero
0,9	Prácticamente verdadero
1	Verdadero

Cuadro 2.1: Tabla endecadaria de grados de verdad.

Para los tensores causa-causa y efecto-efecto, cada matriz en T_k es cuadrada y reflexiva, es decir, todos los elementos en la diagonal principal son iguales a uno, pero no necesariamente simétricos. Además, los elementos fuera de la diagonal están en el intervalo $[0, 1]$, como se indica en la Tabla 2.1, que muestra la tabla endecadaria utilizada en entrevistas de expertos para identificar los grados de verdad (Kaufmann & Aluja, 1993, p. 26). Resumiendo, tenemos que:

$$T_{k,c,c} = 1 \quad \forall c \in \{1, \dots, c\}, \quad T_{k,c,e} \in [0, 1] \quad (2.1)$$

$$T_{k,e,e} = 1 \quad \forall e \in \{1, \dots, e\}, \quad T_{k,c,e} \in [0, 1] \quad (2.2)$$

Por otro lado, el tensor que modela la incidencia causa-efecto no es necesariamente cuadrado, ya que el número de causas y efectos puede diferir. Al igual que los otros tensores, sus valores están en el intervalo $[0, 1]$ y representan el grado de incidencia entre la causa c y el efecto e :

$$T_{k,c,e} \in [0, 1] \quad \forall c \in \{1, \dots, c\}, e \in \{1, \dots, e\} \quad (2.3)$$

$$\left[\begin{array}{cccc|cccc} T_{k,1,1} & T_{k,1,2} & \cdots & T_{k,1,c} & T_{k,1,1} & T_{k,1,2} & \cdots & T_{k,1,e} \\ T_{k,2,1} & T_{k,2,2} & \cdots & T_{k,2,c} & T_{k,2,1} & T_{k,2,2} & \cdots & T_{k,2,e} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ T_{k,c,1} & T_{k,c,2} & \cdots & T_{k,c,c} & T_{k,c,1} & T_{k,c,2} & \cdots & T_{k,c,e} \\ \hline 0 & 0 & \cdots & 0 & T_{k,1,1} & T_{k,1,2} & \cdots & T_{k,1,e} \\ 0 & 0 & \cdots & 0 & T_{k,2,1} & T_{k,2,2} & \cdots & T_{k,2,e} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & T_{k,e,1} & T_{k,e,2} & \cdots & T_{k,e,e} \end{array} \right]$$

Cuadro 2.2: Grafo bipartito encadenado

Durante el desarrollo de este trabajo, los métodos utilizados pueden aplicarse tanto a grafos completos, en los que las causas y los efectos están configurados por el mismo conjunto como a grafos bipartitos encadenados. Construidos a partir de 2.1, 2.2 y 2.3.

En el caso de los grafos bipartitos encadenados, se construye un nuevo grafo completo representado en el Cuadro 2.2 a partir de las relaciones causa-causa (esquina superior izquierda, $T_{k,c,c}$), causa-efecto (esquina superior derecha, $T_{k,c,e}$) y efecto-efecto (esquina inferior derecha, $T_{k,e,e}$) y dado que no existen relaciones efecto-causa, las aristas correspondientes a la esquina inferior izquierda serán iguales a cero.

2.3. Efectos Olvidados

La Teoría de los Efectos Olvidados fue propuesta por Kaufmann y Gil Aluja (1988) para identificar efectos indirectos a partir de procesos causa-efecto.

Para el cálculo de efectos olvidados nos referenciaremos en lo publicado en el artículo de (Chávez-Bustamante et al., 2023, p. 7) que dice que si se construye una matriz de efectos directos E_t (a partir de las matrices de incidencia vistas anteriormente), la matriz de efectos de segundo orden, E_t^2 , se obtiene utilizando convolución max-min, que combina los efectos directos para identificar incidencias indirectas. La convolución max-min de la matriz se calcula como:

$$E_t^{1+2} = E_t \circ E_{t'} \quad (2.4)$$

donde $\circ$ representa la operación de convolución max-min. Los elementos de la matriz de convolución E_t^{1+2} se definen como:

$$\mu_{i,k}^{E_t^{1+2}} = \max \left(\min_{j=1,\dots,n} (\mu_{i,j}^{E_t}, \mu_{j,k}^{E_t}) \right) \quad (2.5)$$

La matriz de efectos de segundo orden se calcula restando la matriz original de efectos directos, E_t , de la matriz convulsionada:

$$E_t^2 = E_t^{1+2} - E_{t'} \quad (2.6)$$

Para calcular efectos de orden superior, se extiende la convolución max-min de forma iterativa:

$$E_t^{1+2+\dots+\eta} = E_t^{1+2+\dots+\eta-1} \circ E_t \quad (2.7)$$

Donde los elementos de la matriz de convolución $E_t^{1+2+\dots+\eta}$ se definen como:

$$\mu_{i,k}^{E_t^{1+2+\dots,\eta}} = \max \left(\min_{j=1,\dots,n} \left(\mu_{i,j}^{E_t^{1+2+\dots,\eta-1}}, \mu_{j,k}^{E_t} \right) \right) \quad (2.8)$$

La matriz de efectos de orden η se obtiene como:

$$E_t^\eta = E_t^{1+2+\dots+\eta} - E_t^{1+2+\dots+\eta-1} \quad (2.9)$$

Entonces, los efectos de orden η son aquellos para los cuales:

$$\mu_{i,k}^{E_t^\eta} - \mu_{i,k}^{E_t^{\eta-1}} > \epsilon \quad (2.10)$$

donde ϵ es un umbral mínimo dado para considerar que el efecto indirecto es mayor que el efecto directo.

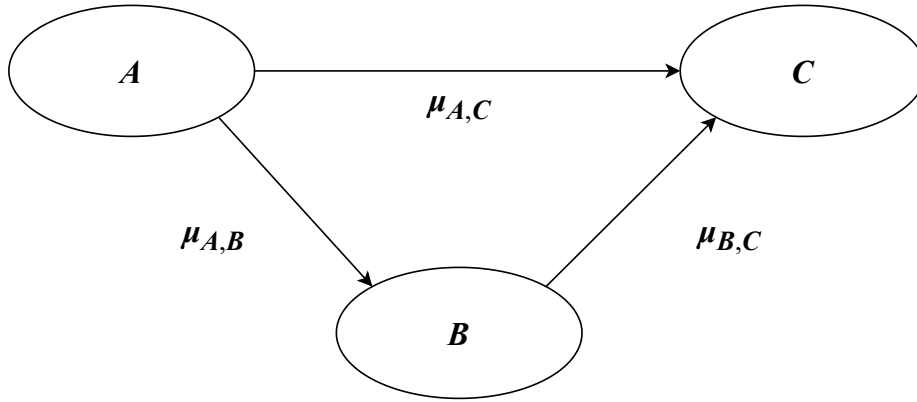


Figura 2.1: Ejemplo de relaciones directas e indirectas.

Por ejemplo, en la Figura 2.1 se muestra un efecto olvidado sencillo con relación indirecta entre dos nodos de una red. Dado tres nodos A , B , C de un grafo, y el peso de cada arista $\mu_{A,B}$, $\mu_{A,C}$ y $\mu_{B,C} \in [0, 1]$, si $\min(\mu_{A,B}, \mu_{B,C}) - \mu_{A,C} > 0,5$, es posible decir que la incidencia indirecta $A \rightarrow C$ por medio de B es más fuerte que la incidencia directa $A \rightarrow C$.

Donde si tomamos la Figura anterior 2.1 y apilamos los grafos de relaciones podríamos representar la información de cada experto y así obtener una estructura de datos que representa la variabilidad del grupo de expertos. La estructura se modela en la Figura 2.2.

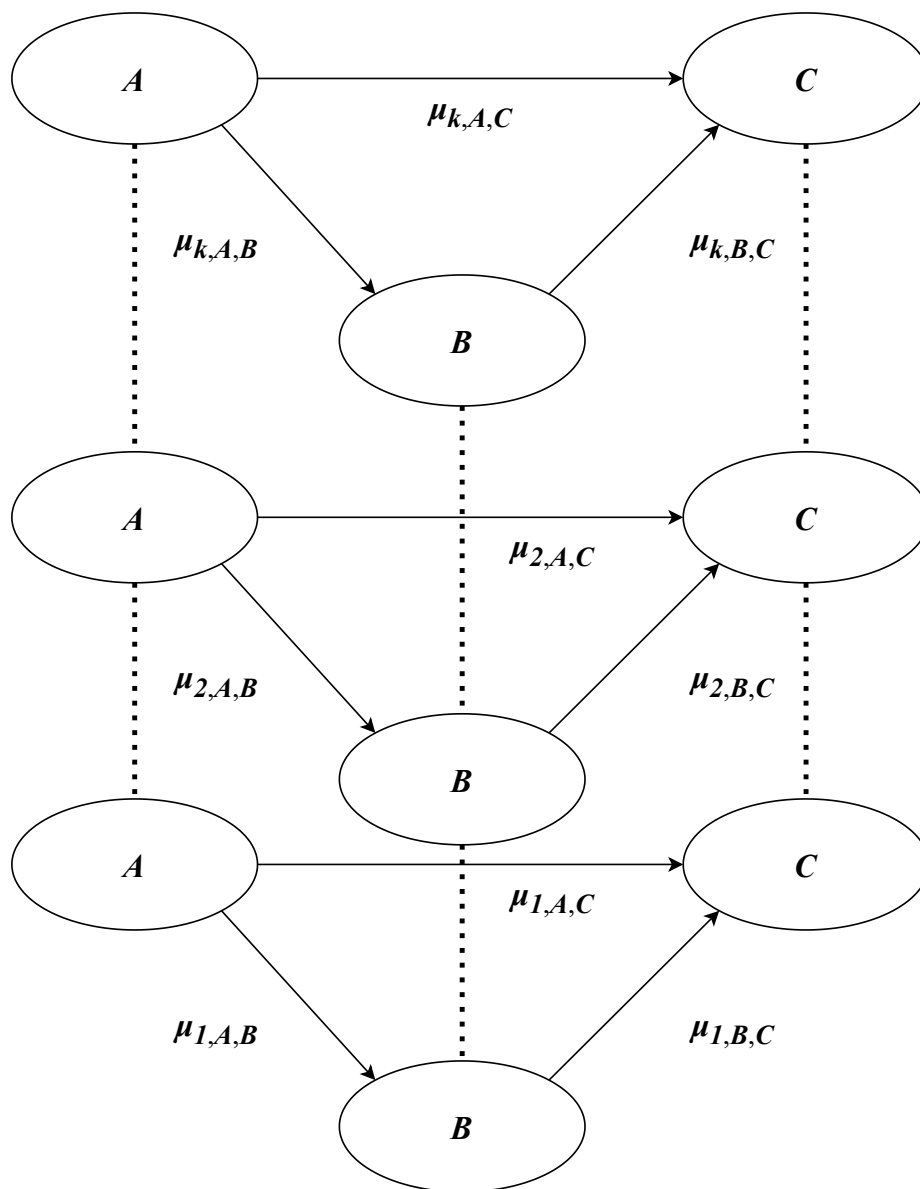


Figura 2.2: Ejemplo de relaciones directas e indirectas con múltiples expertos.

2.4. Efectos Directos

Consideremos un tensor $T_{k,c,e}$, que representa la incidencia entre una causa c y un efecto e , evaluada por los expertos k . A partir de este tensor, definimos el vector de incidencias $\mu_k = (\mu_{k,c,e})$ para todos los expertos $k = 1, \dots, K$, el cual agrupa las evaluaciones realizadas entre c y e .

Para evaluar la significancia de las incidencias entre c y e , planteamos un contraste estadístico unilateral a la izquierda, definido como:

$$H_0 : \bar{\mu} = \varepsilon,$$

$$H_1 : \bar{\mu} < \varepsilon,$$

donde $\bar{\mu}$ denota el promedio de las incidencias calculado a partir del vector μ_k , y ε es el umbral mínimo (o grado de verdad) a partir del cual una incidencia se considera significativa.

El contraste se lleva a cabo mediante una prueba t basada en remuestreo bootstrap (Davison & Hinkley, 1997), acompañada de intervalos de confianza contruidos a partir de los percentiles de las réplicas bootstrap (Altman et al., 2000). Este enfoque se justifica por la convergencia de la distribución de los promedios de las muestras bootstrap hacia una distribución normal, en virtud del teorema del límite central.

El teorema del límite central establece que la distribución de la media muestral converge a una distribución normal cuando el tamaño de las muestras es suficientemente grande, independientemente de la distribución original. En este caso con bootstrap, las medias $\bar{\mu}$ de las réplicas generadas siguen una distribución aproximadamente normal al aumentar el número de réplicas, justificando así el uso de la prueba t e intervalos de confianza.

Como criterio de decisión, cualquier incidencia para la cual se rechace la hipótesis nula será interpretada como no significativa basado en el marco del Cuadro 2.1, ya que su valor promedio se considera estadísticamente inferior al umbral ε , y en consecuencia, podría considerarse como cero.

2.5. Validaciones

Aquí se describen los métodos utilizados para evaluar la correcta implementación de las funciones de `forgeffects` comparadas con las de su implementación anterior `foRgotten`. Ambas funciones generan resultados relacionados con efectos directos y efectos olvidados, los cuales se validarán utilizando pruebas estadísticas que tienen como objetivo evaluar la consistencia y precisión de los resultados obtenidos.

Normalización de los datos

Antes de realizar cualquier análisis, los datos observados y esperados son normalizados para garantizar que sean comparables. Este paso asegura que las comparaciones sean válidas dentro de un marco uniforme y evita sesgos debidos a diferentes escalas de magnitud.

Este procedimiento se aplica tanto a las frecuencias de aparición de los efectos olvidados como a los p -valores utilizados en la evaluación de los efectos directos.

2.5.1. Evaluación y resultados esperados

Tanto los efectos directos como los efectos olvidados calculados por las funciones `foRgotten` y `forgeffects` se evalúan utilizando los siguientes métodos estadísticos:

Pruebas de normalidad: Test de Shapiro-Wilk

El test de Shapiro-Wilk se emplea para evaluar si las diferencias entre los valores observados y esperados siguen una distribución normal. Este test calcula un estadístico W , que mide la correlación entre los datos y una distribución normal ideal. La hipótesis nula (H_0) del test establece que los datos provienen de una población con distribución normal.

Si el valor p asociado al estadístico W es mayor que un nivel de significancia predefinido (por ejemplo, $\alpha = 0,05$), no se rechaza H_0 . Esto sugiere que las diferencias observadas pueden considerarse normalmente distribuidas, lo cual garantiza la aplicabilidad de las pruebas estadísticas posteriores, como la prueba t . Es entonces, que en esta etapa se espera que las diferencias sigan una distribución normal.

Pruebas t pareadas

La prueba t pareada se utiliza para determinar si las diferencias entre los valores observados y esperados tienen una media igual a 0. Esto permite evaluar si existen discrepancias significativas entre los resultados generados por ambas funciones.

- La hipótesis nula (H_0) plantea que la media de las diferencias es igual a 0.
- Si el valor p obtenido en la prueba es mayor que el nivel de significancia (α), no se rechaza H_0 . Esto indica que las diferencias observadas entre los valores normalizados podrían explicarse por variaciones aleatorias y no por discrepancias sistemáticas.

Tamaño del efecto: *Cohen's d*

El tamaño del efecto es una medida de la magnitud de las diferencias observadas, independientemente del tamaño de la muestra. En este caso, se utiliza *Cohen's d* para cuantificar el tamaño del efecto de las diferencias entre los valores observados y esperados.

- *Cohen's d* se calcula como la razón entre la media de las diferencias ($\bar{d}$) y su desviación estándar (s_d):

$$d = \frac{\bar{d}}{s_d}.$$

- Interpretación estándar de *Cohen's d*:
 - $d \approx 0$: Diferencias despreciables.
 - $d = 0,2$: Tamaño del efecto pequeño.
 - $d = 0,5$: Tamaño del efecto mediano.
 - $d = 0,8$: Tamaño del efecto grande.
- En esta validación, se espera que d sea cercano a 0, lo que indica que las diferencias entre los valores observados y esperados son despreciables en magnitud y no relevantes desde un punto de vista práctico.

Capítulo 3

Diagnóstico del problema

Para poner a prueba el algoritmo de `foRgotten` en R, específicamente los de efectos olvidados, se hicieron las pruebas utilizando como parámetros los presentados en el Cuadro 3.1 en un computador descrito en el Cuadro 3.2 realizando tres ejecuciones por cada función para promediar los tiempos de ejecución de cada uno en segundos.

Parámetros <code>foRgotten::FE()</code>	Descripción/Valor
CC	Matriz Causa-Causa 10x16x16
CE	Matriz Causa-Efecto 10x16x4
EE	Matriz Efecto-Efecto 10x4x4
mode	Empirical
thr	0.5
maxOrder	3
reps	$2^{10}, 2^{11}, 2^{12}, 2^{13}, 2^{14}$

Cuadro 3.1: Valores de parámetros para las pruebas de rendimiento en la función de efectos olvidados en `foRgotten`

Memoria RAM	
Cantidad	2 x 8 GB RAM
Velocidad de memoria	2666 MHz
Tipo de Memoria	DDR4
CPU	
Modelo de CPU	AMD Ryzen 7 5700X
Cantidad de núcleos	8
Cantidad de hilos	16
Base Clock (GHz)	3.40
Cache L3 (MB)	32
GPU	
Modelo GPU	NVIDIA GeForce RTX 3060 Ti
Cuda Cores	4864
Tensor Cores	152
RT Cores	38
SM	38
Base Clock (MHz)	1410
Memoria	8 GB GDDR6
Velocidad de Memoria	14 Gbps
Ancho de Banda de Memoria	448 GB/s
Software	
Sistema Operativo	Ubuntu 22.04.5 LTS de 64 bits

Cuadro 3.2: Especificaciones de hardware y software utilizados para este trabajo

Reps	Tiempo Promedio (segundos)
2^{10}	53.378
2^{11}	153.167
2^{12}	442.315
2^{13}	1403.2
2^{14}	4865.967

Cuadro 3.3: Resultados pruebas de rendimiento en foRgotten.

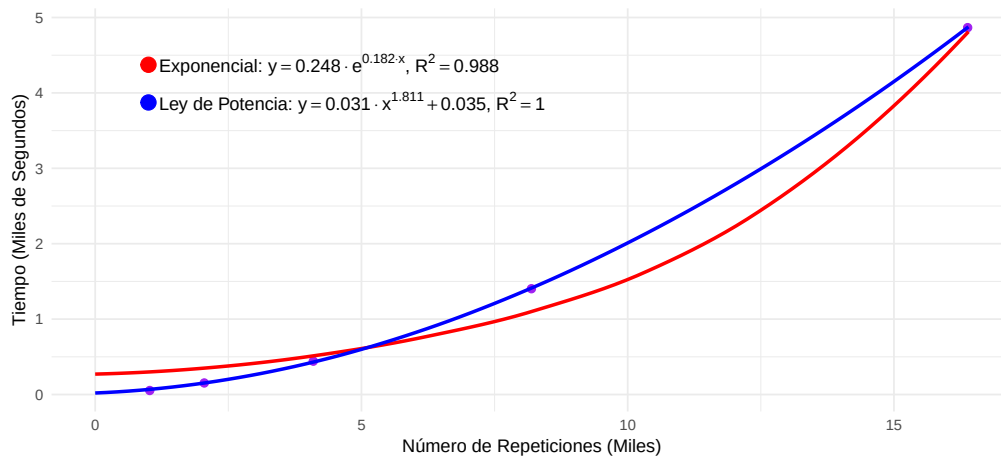


Figura 3.1: Gráfico de rendimiento de efectos olvidados en foRgotten.

3.1. Rendimiento paquete foRgotten

Con base en lo observado en la Figura 3.1, el modelo basado en la ley de potencia describe de mejor manera el comportamiento observado, destacando una relación precisa entre las variables, reflejada en un mayor R^2 en comparación con el modelo exponencial.

El exponente $b = 1,811$ refleja un crecimiento superlineal en los tiempos de ejecución con respecto al número de repeticiones (x), aproximándose a un comportamiento cuadrático ($b \approx 2$). Esto indica que los tiempos de

procesamiento aumentan de forma acelerada, similar a la complejidad $O(n^2)$ observada en algoritmos cuadráticos.

En la práctica, este comportamiento refuerza la importancia de considerar estrategias de optimización, especialmente en escenarios con un alto número de repeticiones empíricas, donde los tiempos de ejecución pueden volverse prohibitivos y comprometer la escalabilidad del sistema en contextos con altos requerimientos.

Capítulo 4

Diseño

4.1. Estructura del paquete

El paquete **forgeeffects**¹ está implementado en Python y se encuentra disponible en PyPI. Utiliza principalmente TensorFlow para el manejo de tensores y ofrece una solución para trabajar con múltiples expertos en matrices de incidencia tridimensionales, facilitando el análisis y cálculo de efectos olvidados.

Las dos funciones principales del paquete son:

- `FE()`: destinada al cálculo de efectos olvidados.
- `directEffects()`: enfocada en el cálculo de indicadores de efectos directos.

Ambas funciones reciben como parámetros matrices tridimensionales en formato Numpy: CC, CE y EE. Estas matrices deben tener la estructura (expertos, filas, columnas) y representan:

- CC: matriz de incidencia causa-causa.
- CE: matriz de incidencia causa-efecto.
- EE: matriz de incidencia efecto-efecto.

¹<https://pypi.org/project/forgeeffects/>

Para resolver problemas basados en grafos completos, es suficiente utilizar los parámetros CC o EE. Sin embargo, para trabajar con grafos bipartitos encadenados, es necesario emplear los tres parámetros CC, CE y EE.

Además, ambas funciones permiten agregar nombres a las causas y efectos mediante arreglos o tuplas de tipo string. Si no se especifican, las causas se representarán por la letra a , con notación $a_1, a_2, \dots, a_n$, donde n es la cantidad de causas evaluadas. Los efectos se representarán con la letra b , usando la notación $b_1, b_2, \dots, b_m$, donde m es la cantidad de efectos evaluados.

4.1.1. Cálculo de efectos olvidados

La función `FE()` está diseñada para calcular los efectos olvidados utilizando datos proporcionados por múltiples expertos. Para ello, se implementa una solución basada en tensores que permite identificar relaciones indirectas entre causas y efectos a través de diferentes órdenes. Esta función utiliza convoluciones max-min iterativas para analizar grafos bipartitos encadenados o grafos completos, considerando un umbral mínimo de significancia (THR) y un orden máximo de exploración (maxorder).

Principales características:

- Genera un grafo bipartito encadenado o un grafo completo para cada experto, remuestreando los valores de los tensores ingresados mediante distribuciones empíricas.
- Identifica relaciones indirectas de diferentes órdenes mediante convoluciones max-min, filtrando únicamente aquellas que superan el umbral establecido.
- Construye rutas intermedias dinámicas entre causas y efectos, indicando los intermediarios en cada orden de efecto olvidado.

Parámetros:

- CC: Matriz de incidencia tridimensional para las relaciones causa-causa.

- CE: Matriz de incidencia tridimensional para las relaciones causa-efecto.
- EE: Matriz de incidencia tridimensional para las relaciones efecto-efecto.
- causes: Permite asignar nombres personalizados a las causas en forma de una lista o tupla de cadenas de texto. (Opcional)
- effects: Permite asignar nombres personalizados a los efectos en forma de una lista o tupla de cadenas de texto. (Opcional)
- THR: Define el umbral mínimo de grado de verdad, dentro del rango $[0, 1]$, para considerar significativas las relaciones indirectas. Por defecto, $\text{THR} = 0.5$.
- maxorder: Establece el orden máximo de los efectos olvidados a calcular. Por defecto, $\text{maxorder} = 2$.
- rep: Define el número de réplicas para el remuestreo empírico.
- device: Permite trabajar tanto con CPU como GPU.

La función `FE()` retorna una lista de *dataframes* que incluye:

- **From:** Origen de las relaciones de efectos olvidados.
- **Through_x:** Campos dinámicos que representan los intermediarios de las relaciones indirectas. Para un efectos de orden n se generan intermediarios que van desde **Through₁** hasta **Through_{n-1}**.
- **To:** Destino de las relaciones de efectos olvidados.
- **Count:** Número de veces que un efecto olvidado se repite entre diferentes expertos.
- **Mean:** Promedio de los valores asociados a las relaciones de efectos olvidados.
- **SD:** Desviación estándar de los valores de las relaciones.

Para utilizar la función de efectos olvidados con GPU, es necesario contar con tarjetas NVIDIA y, preferentemente, tener instalado el NVIDIA Container Toolkit (Corporation, 2024). Este toolkit facilita la instanciación de contenedores que permiten el uso de GPU de manera más sencilla.

4.1.2. Cálculo de efectos directos

Para el cálculo de los efectos directos, la función `directEffects()` estima la incidencia media, el intervalo de confianza unilateral por la izquierda y el valor p , aplicando pruebas t con bootstrap. Esta función es aplicable tanto a grafos completos como a grafos bipartitos encadenados.

Parámetros:

- CC: Matriz de incidencia tridimensional para las relaciones causa-causa.
- CE: Matriz de incidencia tridimensional para las relaciones causa-efecto.
- EE: Matriz de incidencia tridimensional para las relaciones efecto-efecto.
- causes: Permite asignar nombres personalizados a las causas en forma de una lista o tupla de cadenas de texto. (Opcional)
- effects: Permite asignar nombres personalizados a los efectos en forma de una lista o tupla de cadenas de texto. (Opcional)
- THR: Define el umbral mínimo de significancia para las incidencias, dentro del rango $[0, 1]$. Solo las incidencias con un valor igual o mayor a `thr` se consideran significativas para las pruebas. Por defecto, `THR = 0.5`.
- `conf_level`: Establece el nivel de confianza para los intervalos. Por defecto, `conf_level = 0.95`.
- `reps`: Determina el número de réplicas bootstrap utilizadas para el cálculo.

La función devuelve una tabla en formato *dataframe* de datos estructurados con la siguiente información:

- **From:** Origen de la incidencia.
- **To:** Destino de la incidencia.
- **Mean:** Media de la incidencia a través de los expertos.
- **UCI:** Límite superior del intervalo de confianza.
- **p.value:** Valor p calculado.

4.1.3. Conjuntos de datos incluidos en el paquete

El paquete **forgeeffects** incluye tres matrices de incidencia tridimensionales: CC, CE y EE, utilizadas en el estudio «Aplicación de la Teoría de los Efectos Olvidados para Evaluar la Política Pública sobre Contaminación del Aire en la Comuna de Valdivia, Chile» de (Manna et al., 2018).

Los datos corresponden a dieciséis incentivos, cuatro comportamientos y diez informantes clave. Cada informante proporcionó datos con valores acorde a lo estipulado en la Tabla 2.1. Estas matrices pueden cargarse utilizando la función `load_test_data()`, que permite acceder a los archivos `.npy` incluidos en el directorio de datos del paquete.

Ejemplo de uso:

```
import forgeeffects as fe

CC = fe.load_test_data("CC.npy")
CE = fe.load_test_data("CE.npy")
EE = fe.load_test_data("EE.npy")
```

4.1.4. Bibliotecas utilizadas

Además de TensorFlow para el manejo de tensores, se emplean las siguientes bibliotecas que trabajan principalmente con CPU:

- **Pandas** (pandas development team, 2020): para la manipulación de *dataframes*.
- **Numpy** (Harris et al., 2020): para el manejo de los datos de entrada en ambas funciones y para procesar datos al armar caminos de orden superior en la función de efectos olvidados.

Los principales requerimientos del paquete son:

- **Python** ≥ 3.9 y ≤ 3.11
- **Pandas** ≥ 1.0
- **NumPy** ≥ 1.18
- **TensorFlow** 2.13
- **TensorFlow Probability** 0.20.0

El paquete está licenciado bajo la **Licencia MIT** y es independiente del sistema operativo.

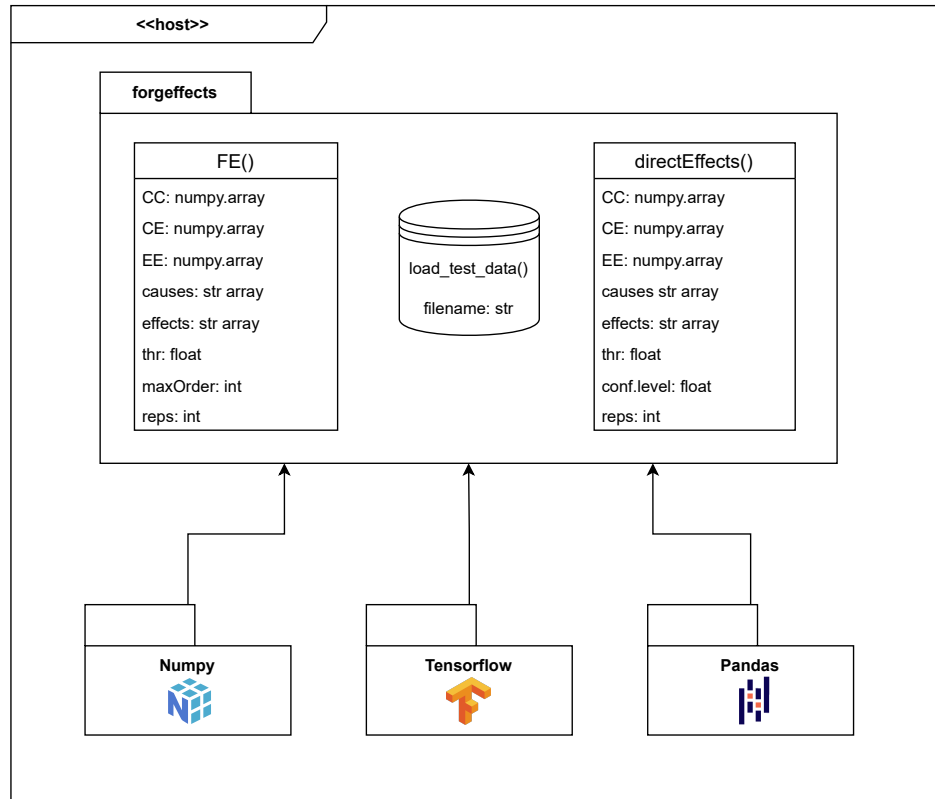


Figura 4.1: Representación gráfica de estructura de forgeeffects.

Capítulo 5

Implementación

5.1. Función de efectos olvidados - FE()

Para calcular efectos olvidados a partir de información proporcionada por múltiples expertos, se implementó un algoritmo iterativo basado en tensores, donde todas las convoluciones max-min se realizan en un solo cálculo a través de los órdenes de efectos olvidados identificados. Considerando un umbral mínimo de grado de verdad ϵ y un orden máximo de exploración definido por el usuario (*maxOrder*), el algoritmo procede de la siguiente manera:

Inicialmente, se utilizan los tensores de incidencia $\mathbf{CC}_k$, $\mathbf{CE}_k$ y $\mathbf{EE}_k$ para cada experto k , según las ecuaciones (2.1), (2.2) y (2.3). A partir de estos, se genera un nuevo grafo bipartito encadenado, o bien, un grafo completo que llamaremos $\mathbf{T}_k$, del cual se crean nuevos expertos remuestreando los valores del grafo utilizando una distribución empírica con cuantiles aleatorios.

Después, en el Algoritmo 1, se procede a calcular el tensor conjugado de orden n , denotado como $\mathbf{T}_k^{\sum_{i=1}^n i}$, que contiene todos los efectos directos e indirectos de orden n , calculado mediante la convolución max-min presentada en la ecuación (2.7). Luego, para obtener solo los efectos indirectos de orden n , se sustrae el tensor conjugado $T_{k_1 \dots k_s}^{\sum_{i=1}^{n-1} i}$ del tensor conjugado $T_{k_1 \dots k_s}^{\sum_{i=1}^n i}$ para obtener así el tensor $\mathbf{T}_k^n$, como se describe en la ecuación (2.9).

Una vez obtenido el tensor con los valores de los efectos olvidados, en una función llamada *Indexes* se filtran los valores del tensor $\mathbf{T}_k^n$ que sean mayores al umbral ϵ , como se indica en la ecuación (2.10), generando un nuevo tensor que llamaremos $\mathbf{Edges}_k^n$, que contiene los caminos de los efectos olvidados de orden n con estructura $(k, \text{origen}, \text{intermediario}, \text{destino})$, y otro tensor que llamaremos $\mathbf{values}^n$ que contiene los valores de cada camino.

Para la obtención de $\mathbf{Edges}_k^n$ lo que se hace es tomar los máximos de los mínimos calculados anteriormente en la convolución max-min (2.8) e identificar los índices por los cuales pasaron los máximos para obtener así los intermediarios de cada par origen-destino de aquellos valores de $\mathbf{T}_k^n$ mayores al umbral ϵ .

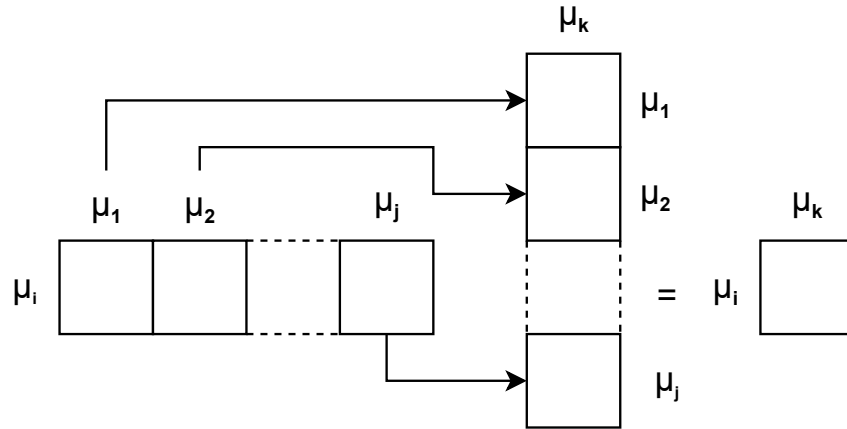


Figura 5.1: Esquema de convolución max-min (Kaufmann & Gil Aluja, 1988, p. 28)

Para visualizar mejor la obtención de **Edges**, podemos utilizar el esquema de la Figura 5.1 el cual representa la convolución max-min que se calcula como el máximo de los mínimos entre filas y columnas. Si definimos los símbolos operativos $\wedge$ como el mínimo entre dos números y $\vee$ como el máximo entre los valores a considerar podemos decir que:

$$\mu_{i,k} = \vee(\mu_{i,j} \wedge \mu_{j,k}) \quad (5.1)$$

Donde, por cada $\mu_{i,j} \wedge \mu_{j,k}$ que represente el o los máximos del conjunto de mínimos, se obtendrán los caminos de **Edges** como $\mu_{i,j,k}$, los cuales representan lo mencionado anteriormente como (origen,intermediario,destino).

En el algoritmo, mediante un tensor llamado **min_result**_kⁿ, que corresponde a los mínimos del resultado de la convolución de orden n , se extraen únicamente los mínimos de aquellos valores que en **T**_kⁿ fueron mayores al umbral ϵ . Posteriormente, se crea una máscara booleana con este tensor de mínimos para identificar aquellos valores máximos y determinar si existe uno o más caminos intermedios en un mismo par origen-destino. Finalmente, se concatenan los intermediarios encontrados con sus índices correspondientes, generando así el tensor **Edges**_kⁿ.

Cuando se identifican efectos olvidados de orden superior, se procede a lo realizado en el Algoritmo 2, que es construir los caminos correspondientes. Para ello, se comparan los pares origen-destino de **Edges**_kⁿ⁻¹ con los pares origen-intermediario de los caminos **Edges**_kⁿ. Si coinciden, se forman los caminos de los efectos olvidados que incluyen las rutas intermedias del orden correspondiente ($k, \text{origen}, j_1, j_2, \dots, j_n, \text{destino}$).

Este proceso se repite iterativamente hasta que el tensor **Edges**_kⁿ es un tensor de ceros o se alcanza el orden máximo (*maxOrder*). Cuando esto ocurre, el algoritmo se detiene y retorna un dataframe con los efectos olvidados encontrados hasta el orden n , donde se procede a contar la cantidad de apariciones de caminos iguales entre distintos efectos encontrados de otros expertos, junto a la media de los valores de los caminos y su desviación estándar.

Mediante este enfoque tensorial, el algoritmo es capaz de identificar eficientemente los efectos olvidados de órdenes superiores, realizando las convoluciones max-min y actualizando los caminos a medida que se encuentran nuevos efectos.

Algorithm 1 Iterative maxmin

```

1: procedure ITERATIVE MAXMIN( $\mathbf{T}_k, \varepsilon \in [0, 1], \text{maxOrder} \geq 2$ )
2:   original_tensor  $\leftarrow T_k$ 
3:   gen_tensor  $\leftarrow T_k$ 
4:   Initialize empty lists:
5:     result_paths, result_values
6:   for  $n \leftarrow 2$  to  $\text{maxOrder} + 1$  do
7:      $\min\_result_k^n, T_k^{\sum_{i=1}^n i} \leftarrow \text{maxmin}(\text{gen\_tensor}, \text{original\_tensor})$ 
8:      $T_k^n \leftarrow T_k^{\sum_{i=1}^n i} - \text{gen\_tensor}$ 
9:      $\text{Edges}_k^n, \text{values}^n \leftarrow \text{indexes}(\min\_result_k^n, T_k^n, \varepsilon)$ 
10:    if  $\text{Edges}_k^n$  is empty then
11:      Break loop
12:    end if
13:    if  $n \leftarrow 2$  then
14:      Append  $\text{Edges}_k^n$  to result_paths
15:      Append  $\text{values}^n$  to result_values
16:    else
17:      [paths, values]  $\leftarrow \text{createPaths}(\text{Edges}_k^{n-1}, \text{Edges}_k^n, \text{values}^n)$ 
18:      Append paths to result_paths
19:      Append values to result_values
20:    end if
21:    gen_tensor  $\leftarrow T_k^{\sum_{i=1}^n i}$ 
22:  end for
23:  Return result_paths, result_values
24: end procedure

```

Algorithm 2 Create Paths

```

1: procedure CREATEPATHS( $\text{Edges}_k^{n-1}, \text{Edges}_k^n, \text{values}^n$ )
2:   Extract prev_edges  $\leftarrow$  columns:  $k, \text{origin}, \text{destination}$  of  $\text{Edges}_k^{n-1}$ 
3:   Extract edges  $\leftarrow$  columns:  $k, \text{origin}, \text{intermediary}$  of  $\text{Edges}_k^n$ 
4:   Identify unique values in first column of prev_edges and edges
5:   Find common values in first column of both tensors
6:   Initialize empty lists:
7:     paths_list, matched_values
8:   for each common value do
9:     Filter rows in prev_edges and edges where first column
    equals current value
10:    Check matches  $\leftarrow$  between rows in prev_edges and edges
11:    if matches  $\leftarrow 0$  then
12:      Continue
13:    end if
14:    new_path  $\leftarrow$  destination column from edges at the end of
    prev_edges
15:    new_values  $\leftarrow$  matches values found in edges
16:    Append new_path to paths_list
17:    Append new_values to matched_values
18:  end for
19:  Return paths_list and matched_values
20: end procedure

```

5.2. Función de efectos directos - `directEffects()`

Para calcular los efectos directos, se realiza un procedimiento basado en el enfoque de remuestreo bootstrap implementado en la función `boot.t.test` de la librería `MKinfer` (Kohl, 2024) en R. Este método evalúa las diferencias de medias ajustadas y genera un intervalo de confianza mediante el uso de percentiles sobre distribuciones generadas por remuestreo. En esta implementación, el cálculo ha sido adaptado para utilizarse con tensores tridimensionales utilizando la biblioteca TensorFlow y sigue los siguientes pasos:

1. Se calcula la media del tensor a lo largo del eje que corresponde a los expertos. Esta media representa la incidencia promedio entre cada causa y efecto.
2. Los datos son centrados restando esta media del tensor original, generando un nuevo tensor centrado.
3. Se realiza un remuestreo bootstrap generando índices aleatorios que seleccionan subconjuntos del tensor centrado.
4. Para cada réplica bootstrap, se calcula la incidencia promedio entre causas y efectos, considerando todas las muestras seleccionadas.
5. Con estas réplicas, se calcula un estadístico t que compara las medias bootstrap con un umbral predefinido. Esto se realiza tanto para las réplicas bootstrap como para la muestra original.
6. Finalmente, se evalúa cuántas réplicas cumplen con la condición de que el estadístico t de las réplicas sea menor que el estadístico de la muestra original. Esto determina el valor p asociado, que refleja la significancia estadística de las incidencias evaluadas.

La comparación entre la implementación en R y su adaptación en Python puede observarse en la Tabla 5.1. Aquí se destacan las diferencias en la estructura de los datos y las limitantes que se tienen ya que lo realizado en Python fue adaptado únicamente para el cálculo de efectos directos.

Aspecto	Código en R (<code>boot.t.test</code>)	Código en Python
Entorno	Específico de R, manipula matrices base y cálculos con funciones propias del lenguaje.	Utiliza TensorFlow, optimizado para cálculos tensoriales.
Datos de entrada	Acepta vectores o matrices como entrada (x , y). Soporta datos pareados, diferencia de medias o una sola muestra.	Acepta tensores tridimensionales, solo soporta una sola muestra.
Forma de remuestreo	Genera réplicas bootstrap directamente de los datos centrados.	Usa reemplazo con índices aleatorios para remuestrear las matrices centradas.
Cálculo del estadístico t	Realiza el cálculo usando la fórmula estándar, adaptando para una muestra o dos muestras, considerando μ como hipótesis nula.	Cálculo similar pero totalmente tensorizado, limitado a una sola muestra.
Intervalo de confianza	Calcula percentiles directamente en R usando <code>quantile</code> .	Usa <code>tfp.stats.percentile</code> para percentiles directamente en TensorFlow.
Valor p	Calcula p -valores para diferentes hipótesis alternativas (<code>two.sided</code> , <code>less</code> , <code>greater</code>) con control explícito de simetría.	El p -valor se calcula solo para hipótesis unilaterales (menor que).

Cuadro 5.1: Comparación entre `boot.t.test` en R y lo realizado en Python adaptado para efectos directos.

Capítulo 6

Validación

Para validar el algoritmo `forgeffects` y compararlo con `foRgotten`, se realizaron pruebas tanto para las funciones de efectos olvidados como para las de efectos directos. Al igual que en la etapa de diagnóstico, el algoritmo de efectos olvidados de `forgeffects` fue evaluado en cuanto a su velocidad, con el objetivo de calcular los tiempos promedio de ejecución en segundos, utilizando los parámetros descritos en el Cuadro 6.1, utilizando la misma computadora empleada para las pruebas de diagnóstico del Cuadro 3.2.

Una vez realizadas las pruebas, la precisión y consistencia de los resultados se evaluaron mediante pruebas estadísticas aplicadas tanto a los efectos directos como a los efectos olvidados. Para los efectos olvidados, se analizaron las primeras diez muestras de las apariciones de caminos indirectos, utilizando los mismos parámetros descritos en los Cuadros 3.1 y 6.1, con la diferencia de que solo se evaluaron con $rep = 2^{14}$. Las diferencias entre las frecuencias observadas y esperadas fueron analizadas mediante el test de Shapiro-Wilk para comprobar su normalidad, seguido de una prueba t pareada para determinar si dichas diferencias tienen una media igual a 0. Adicionalmente, se calculó el tamaño del efecto mediante *Cohen's d* para evaluar la relevancia de las discrepancias encontradas.

En cuanto a los efectos directos, los p -valores generados se sometieron al mismo procedimiento de análisis estadístico, utilizando los parámetros descritos en el Cuadro 6.6. Este enfoque permitió garantizar que las diferencias entre los resultados obtenidos por `foRgotten` y `forgeffects`

no solo fueran estadísticamente insignificantes, sino también irrelevantes desde un punto de vista práctico, asegurando que ambos algoritmos produjeran resultados coherentes y comparables bajo las mismas condiciones experimentales.

Es importante destacar que esta validación se realizó específicamente para los casos y parámetros evaluados en este estudio. Por tanto, los resultados obtenidos que muestran la ausencia de diferencias significativas entre las frecuencias observadas y esperadas para los efectos olvidados, así como entre los p valores generados para los efectos directos, son válidos únicamente bajo estas condiciones y no deben generalizarse a todas las posibles configuraciones o escenarios. Este análisis confirma entonces la consistencia de ambos algoritmos dentro de los límites establecidos por los parámetros evaluados.

Parámetros forgeffects.FE()	Descripción/Valor
CC	Matriz Causa-Causa 10x16x16
CE	Matriz Causa-Efecto 10x16x4
EE	Matriz Efecto-Efecto 10x4x4
thr	0.5
maxOrder	3
reps	$2^{10}, 2^{11}, 2^{12}, 2^{13}, 2^{14}$

Cuadro 6.1: Valores de parámetros para las pruebas de rendimiento en la función de efectos olvidados en `forgeffects`

Reps	Tiempo Promedio (segundos)
2^{10}	0.244880
2^{11}	0.373535
2^{12}	0.523934
2^{13}	0.982073
2^{14}	1.993936

Cuadro 6.2: Resultados pruebas de rendimiento en `forgeffects` con CPU.

Reps	Tiempo Promedio (segundos)
2^{10}	0.237795
2^{11}	0.354706
2^{12}	0.474169
2^{13}	0.806187
2^{14}	1.505697

Cuadro 6.3: Resultados pruebas de rendimiento en forgeeffects con GPU.

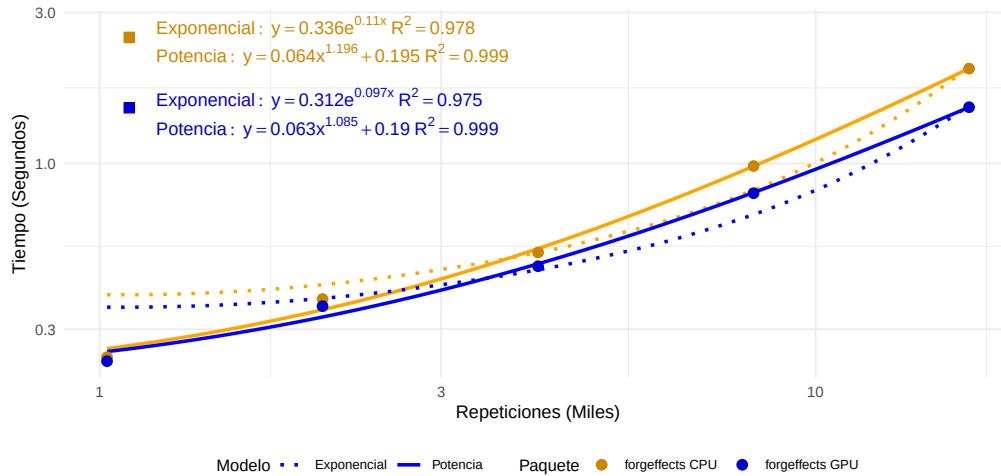


Figura 6.1: Gráfico de rendimiento de efectos olvidados en forgeeffects.

6.1. Rendimiento paquete forgeeffects

Con base en lo observado en la Figura 6.1, el modelo basado en la ley de potencia describe de mejor manera el comportamiento observado en ambas evaluaciones (CPU y GPU). Esto se evidencia por un mayor coeficiente de determinación (R^2) en comparación con el modelo exponencial, lo que refleja una relación más precisa entre las variables.

En el caso de la evaluación en CPU, el exponente $b = 1,196$ de la ley de potencia refleja un crecimiento subcuadrático en los tiempos de ejecución

con respecto al número de repeticiones (x). Aunque el crecimiento es superlineal, es menos acelerado que el de algoritmos con complejidad cuadrática ($O(n^2)$) y sus coeficientes sugieren una relación cercana a la lineal.

Por otro lado, la evaluación en GPU muestra un comportamiento aún más eficiente, con un exponente $b = 1,085$ en la ley de potencia. Esto indica que el crecimiento del tiempo de ejecución está más cercano a ser lineal que la evaluación con CPU, por ende, va mejor en rendimiento.

En la práctica, este comportamiento indica que si bien los tiempos de ejecución entre CPU y GPU no presentan grandes diferencias entre si, el algoritmo de `forgeeffects` presenta una mejor escalabilidad en comparación con los resultados obtenidos en `foRgotten`.

Para visualizar mejor la comparativa entre los algoritmos de `foRgotten` y `forgeeffects`, se puede observar la Figura 6.2.

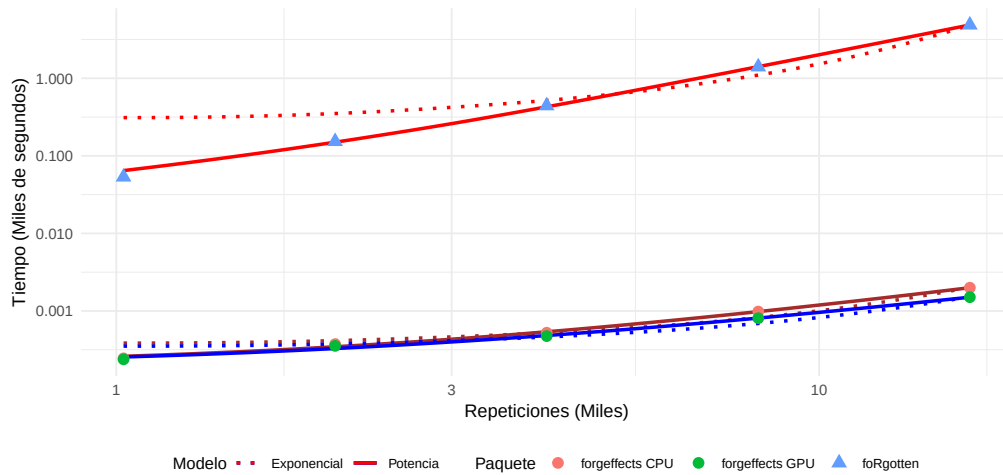


Figura 6.2: Gráfico de rendimiento de efectos olvidados en `foRgotten` y `forgeeffects`.

Record	Dist 1 (foRgotten)	Dist 2 (forgeffects)
a10 → a11 → a14	3559	3525
a13 → a16 → a1	3276	3273
a3 → a6 → a9	3107	3123
a8 → a13 → a10	3096	3069
a14 → a11 → a4	2746	2781
a13 → a6 → a9	2734	2739
a9 → a8 → a13	2714	2637
a7 → a6 → a9	2616	2569
a15 → a11 → a3	2592	2561
a8 → a6 → a9	2591	2619

Cuadro 6.4: Tabla de contingencia de aparición de efectos olvidados de segundo orden entre foRgotten y forgeffects

Record	Dist 1 (foRgotten)	Dist 2 (forgeffects)
b3 → b2 → b4 → b1	21	22
a14 → a7 → a6 → a9	14	11
a9 → a6 → a11 → a14	13	9
a3 → a16 → a11 → a14	12	3
a14 → a7 → a1 → a12	9	3
a14 → a13 → a16 → a1	8	4
a14 → a11 → a6 → a9	8	10
a14 → a2 → a6 → a9	7	7
a14 → a13 → a11 → a3	7	3
b2 → b3 → b4 → b1	7	7

Cuadro 6.5: Tabla de contingencia de aparición de efectos olvidados de tercer orden entre foRgotten y forgeffects

Parámetros <code>directEffects()</code>	Descripción/Valor
EE	Matriz Efecto-Efecto 4x4x10
thr	0.5
conf_level	0.95
reps	10000

Cuadro 6.6: Valores de parámetros para las pruebas de validación en las funciones de efectos directos en `foRgotten` y `forgeffects`.

Record	Dist 1 (<code>foRgotten</code>)	Dist 2 (<code>forgeffects</code>)
b1 → b2	0.92533	0.93398
b1 → b3	0.80976	0.81922
b1 → b4	0.91651	0.92677
b2 → b1	0.75193	0.75904
b2 → b3	0.96651	0.97193
b2 → b4	0.98720	0.98962
b3 → b1	0.69332	0.70139
b3 → b2	0.99971	0.99963
b3 → b4	0.89097	0.89945
b4 → b1	0.91297	0.92757
b4 → b2	0.99664	0.99718
b4 → b3	0.99905	0.99884

Cuadro 6.7: Tabla de contingencia de niveles de significancia (p -valores) en las funciones de efectos directos entre `foRgotten` y `forgeffects`.

6.2. Resultados de efectos olvidados

En esta sección se presentan los resultados obtenidos al comparar las frecuencias de aparición de efectos olvidados de segundo orden y de tercer orden entre las funciones `foRgotten` y `forgeffects`. Los valores

analizados corresponden a las Tablas 6.4 y 6.5, donde se normalizaron las frecuencias y se evaluaron las diferencias entre ambas distribuciones.

6.2.1. Resultados de segundo orden

Para los efectos olvidados de segundo orden, se analizaron las frecuencias presentadas en la Tabla 6.4. Las diferencias entre las distribuciones generadas por `forGotten` y `forEffects` fueron evaluadas con los siguientes resultados:

Normalidad de las diferencias

El test de Shapiro-Wilk se utilizó para comprobar si las diferencias normalizadas seguían una distribución normal, obteniendo:

- Estadístico $W = 0,96006$.
- Valor $p = 0,7865$.

Dado que el valor p es mayor que el nivel de significancia ($\alpha = 0,05$), no se rechaza la hipótesis nula de normalidad. Por lo tanto, las diferencias pueden considerarse normalmente distribuidas.

Prueba t pareada

La prueba t pareada se realizó para determinar si la media de las diferencias es igual a 0. Los resultados obtenidos fueron:

- Estadístico $t = 1,0663 \times 10^{-14}$.
- Grados de libertad (df) = 9.
- Valor $p = 1$.
- Intervalo de confianza al 95 %: $[-0,000883, 0,000883]$.
- Media de las diferencias: $4,163 \times 10^{-18}$.

Dado que $p = 1$, no se encontraron diferencias estadísticamente significativas entre las frecuencias normalizadas generadas por las funciones.

Tamaño del efecto

El tamaño del efecto (*Cohen's d*) fue calculado con el siguiente resultado:

- $d = 3,37183 \times 10^{-15}$.

Un valor de d cercano a 0 indica que las diferencias no son relevantes desde un punto de vista práctico.

6.2.2. Resultados de tercer orden

Para los efectos olvidados de tercer orden, se analizaron las frecuencias presentadas en la Tabla 6.5. Las diferencias entre las distribuciones generadas por `foRgotten` y `forgeffects` fueron evaluadas con los siguientes resultados:

Normalidad de las diferencias

El test de Shapiro-Wilk se utilizó para comprobar si las diferencias normalizadas seguían una distribución normal, obteniendo:

- Estadístico $W = 0,98925$.
- Valor $p = 0,9958$.

El valor p mayor que $\alpha = 0,05$ indica que no se rechaza la hipótesis nula de normalidad. Por lo tanto, las diferencias pueden considerarse normalmente distribuidas.

Prueba t pareada

La prueba t pareada se realizó para determinar si la media de las diferencias es igual a 0. Los resultados obtenidos fueron:

- Estadístico $t = -4,7205 \times 10^{-17}$.
- Grados de libertad (df) = 9.
- Valor $p = 1$.

- Intervalo de confianza al 95 %: $[-0,033285, 0,033285]$.
- Media de las diferencias: $-6,946 \times 10^{-19}$.

El valor $p = 1$ indica que no hay evidencia de diferencias estadísticamente significativas entre las frecuencias normalizadas generadas por las funciones.

Tamaño del efecto

El tamaño del efecto (*Cohen's d*) fue calculado con el siguiente resultado:

- $d = -1,492766 \times 10^{-17}$.

Un valor de d cercano a 0 refuerza la conclusión de que las diferencias no son relevantes desde un punto de vista práctico.

6.3. Resultados de efectos directos

En esta sección se presentan los resultados obtenidos al comparar los niveles de significancia (p -valores) generados por las funciones `foRgotten` y `forgeffects` en las relaciones de efectos directos. Los valores analizados corresponden a la Tabla 6.7, donde se normalizaron los p -valores y se evaluaron las diferencias entre ambas distribuciones.

Normalidad de las diferencias

El test de Shapiro-Wilk se utilizó para comprobar si las diferencias entre los p -valores normalizados seguían una distribución normal. Los resultados obtenidos fueron:

- Estadístico $W = 0,90158$.
- Valor $p = 0,1663$.

Dado que el valor p es mayor que el nivel de significancia ($\alpha = 0,05$), no se rechaza la hipótesis nula de normalidad. Esto indica que las diferencias pueden considerarse normalmente distribuidas, lo cual valida la aplicabilidad de las pruebas estadísticas posteriores.

Prueba t pareada

La prueba t pareada se utilizó para evaluar si la media de las diferencias es igual a 0. Los resultados fueron los siguientes:

- Estadístico $t = 4,3029 \times 10^{-14}$.
- Grados de libertad (df) = 11.
- Valor $p = 1$.
- Intervalo de confianza al 95 %: $[-0,000296, 0,000296]$.
- Media de las diferencias: $5,78242 \times 10^{-18}$.

El valor $p = 1$ sugiere que no hay evidencia de diferencias estadísticamente significativas entre los p -valores generados por `foRgotten` y `forgeffects`.

Tamaño del efecto

El tamaño del efecto (*Cohen's d*) fue calculado con el siguiente resultado:

- $d = 1,242126 \times 10^{-14}$.

Un valor de d extremadamente cercano a 0 indica que las diferencias entre las distribuciones son estadísticamente insignificantes.

Capítulo 7

Trabajo Futuro

7.1. Optimización algoritmo efectos olvidados

Como trabajo futuro, se propone optimizar aún más el algoritmo de efectos olvidados mediante el uso de soluciones basadas en GPU. Aunque esta investigación utilizó TensorFlow para gestionar operaciones tensoriales, se identificaron ciertas limitaciones en su manejo de memoria al trabajar con datos de gran escala y redes más complejas con GPU. En aquellos cálculos donde era necesario realizar múltiples comparaciones en la etapa de armado de caminos, tanto grandes como pequeños, se generaba un overhead significativo en términos de tiempo y recursos computacionales al emplear GPU. Por ello, se sugiere explorar bibliotecas como CUDA (NVIDIA et al., 2020) para aprovechar de manera más eficiente los recursos.

Más a fondo, en este trabajo se identificó que TensorFlow presenta desafíos significativos en la administración de memoria GPU debido a su estrategia predeterminada de asignación, que reserva por completo la memoria disponible, incluso si no se utiliza en su totalidad. Aunque es posible configurar el uso de memoria para que crezca gradualmente (`memory_growth=True`), esta solución no fue lo suficiente para optimizar la etapa de armado de caminos ya que en esta etapa se recurría a la iteración. Operaciones dinámicas que requieren realizar iteraciones sobre conjuntos de datos y comparar múltiples dimensiones, generan fragmentación de memoria si los recursos no se gestionan adecuadamente, dificultando la reutilización eficiente. Por el

contrario, bibliotecas como CUDA permiten un control más granular sobre la asignación y liberación de memoria, superando estas limitaciones mediante un uso más eficiente de los recursos disponibles y un menor overhead computacional por lo que puede llegar a ser una posible solución a este problema.

7.2. Implementación interactiva para usuarios finales

Además de las mejoras técnicas, se planea en un futuro llevar este trabajo a una plataforma web que permita a los usuarios interactuar fácilmente con el sistema para la creación y manipulación de matrices de incidencia. La idea es que, a través de una interfaz gráfica intuitiva, los usuarios puedan definir las matrices de entrada (causa-causa, causa-efecto, efecto-efecto), colaborando entre distintos usuarios o expertos para llevar los datos a un modelo que pueda ajustarse a los parámetros del algoritmo según sus necesidades específicas.

El cálculo de los efectos olvidados se delegaría a una API externa diseñada para manejar estas operaciones y los resultados obtenidos podrían visualizarse directamente en la página web, permitiendo a los usuarios explorar gráficamente las relaciones descubiertas y los análisis de impacto derivados.

Con esto no solo se democratizaría el acceso al algoritmo, sino que también facilitaría su adopción en contextos académicos, empresariales y de investigación, eliminando la necesidad de conocimientos en programación o manejo de herramientas especializadas como Python.

Capítulo 8

Conclusiones

Los resultados de esta investigación destacan que `forgeffects`, implementado en Python, logra reproducir de manera consistente los resultados generados por `foRgotten` en R en los casos evaluados, tanto en la identificación de efectos olvidados como en la evaluación de efectos directos. Su rendimiento subcuadrático y su implementación en un entorno ampliamente utilizado como Python hacen de `forgeffects` una herramienta robusta y escalable.

Esto establece un marco sólido para su uso en aplicaciones prácticas, especialmente en entornos que requieren flexibilidad, eficiencia computacional y reproducibilidad en el análisis de efectos directos y efectos indirectos.

Como último punto, podemos hacer mención a que al estar desarrollado bajo una licencia MIT, se espera que este algoritmo pueda mejorarse o incluso reimplementarse en el futuro por parte de la comunidad. Esto abre la posibilidad de optimizar aún más su rendimiento con las posibles soluciones que se plantearon antes y adaptarlo a nuevos contextos o expandir su aplicabilidad en diversas áreas, garantizando así su evolución continua y su relevancia en el análisis de efectos olvidados.

Bibliografía

- A. Alves, L. G., Mangioni, G., Rodrigues, F. A., Panzarasa, P., & Moreno, Y. (2018). Unfolding the Complexity of the Global Value Chain: Strength and Entropy in the Single-Layer, Multiplex, and Multi-Layer International Trade Networks. *Entropy*, 20(12). <https://doi.org/10.3390/e20120909>
- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., ... Zheng, X. (2015). TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems [Version 2.x].
- Altman, D. G., Machin, D., Bryant, T. N., & Gardner, M. J. (2000). *Statistics with Confidence: Confidence Intervals and Statistical Guidelines*. BMJ Books.
- Chávez-Bustamante, F., Mardones-Arias, E., Rojas-Mora, J., & Tijmes-Ihl, J. (2023). A Forgotten Effects Approach to the Analysis of Complex Economic Systems: Identifying Indirect Effects on Trade Networks. *Mathematics*, 11(3). <https://doi.org/10.3390/math11030531>
- Corporation, N. (2024). *NVIDIA Container Toolkit Documentation* [<https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/>].
- Davison, A. C., & Hinkley, D. V. (1997). *Bootstrap Methods and their Application*. Cambridge University Press.
- Drakopoulos, G., Kanavos, A., Karydis, I., Sioutas, S., & G. Vrahatis, A. (2017). Tensor-Based Semantically-Aware Topic Clustering of Biomedical Documents. *Computation*, 5(3). <https://doi.org/10.3390/computation5030034>

- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
- Kaufmann, A., & Gil Aluja, J. (1988). Modelos para la Investigación de efectos olvidados, Milladoiro. *Santiago de Compostela, España*.
- Kaufmann, A., & Aluja, J. G. (1993). *Técnicas especiales para la gestión de expertos*. Editorial Milladoiro.
- Kohl, M. (2024). *MKinfer: Inferential Statistics* [R package version 1.2]. <https://github.com/stamats/MKinfer>
- Manna, E. M., Rojas-Mora, J., & Colther, C. (2018, enero). Application of the Forgotten Effects Theory for Assessing the Public Policy on Air Pollution of the Commune of Valdivia, Chile. https://doi.org/10.1007/978-3-319-65687-8_6
- Mardones-Arias, E., & Rojas-Mora, J. (2022). *foRgotten* [R package version 1.1.0]. <https://github.com/ElliottMardones/foRgotten>
- NVIDIA, Vingelmann, P., & Fitzek, F. H. (2020). CUDA, release: 10.2.89. <https://developer.nvidia.com/cuda-toolkit>
- pandas development team, T. (2020, febrero). *pandas-dev/pandas: Pandas* (Ver. latest). Zenodo. <https://doi.org/10.5281/zenodo.3509134>
- Wang, Q., Ihme, M., Chen, Y.-F., & Anderson, J. (2022). A TensorFlow simulation framework for scientific computing of fluid flows on tensor processing units. *Computer Physics Communications*, 274, 108292. <https://doi.org/https://doi.org/10.1016/j.cpc.2022.108292>